

Sistemas Distribuídos e Computação em Nuvem

Aula 15 — Observabilidade: enxergar o que acontece no sistema

C3 · Nuvem, implantação e segurança · 19/11/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — C3

C1 (1–7)  · C2 (8–12) 

C3 · **Nuvem, implantação e segurança** (13–18)

Nuvem  · orquestração  · **observabilidade** ·
segurança/LGPD · fechamento.

 **Aula 15.** Seu sistema tem vários serviços. Quando algo fica lento ou quebra, como você **enxerga** onde?

 Com um serviço só, um `print` bastava. Com **vários**, você precisa de **observabilidade**: logs, métricas e **rastreamento** para ver a jornada de uma requisição.

Retomada

 Você subiu o C3.A2 com um comando e publicou a função serverless (3 medições)?

- A stack tem **gateway + inferência + worker + banco**. Um usuário reclama: "**está lento**".
- **Onde** está a lentidão? No gateway? Na busca? Na chamada ao LLM? Sem instrumentação, é **adivinhação**.

Objetivos desta aula

Ao final, você será capaz de:

1. **Diferenciar** os **três pilares** da observabilidade: **logs**, **métricas** e **traces**.
2. **Explicar** por que o **rastreamento distribuído** é essencial em vários serviços.
3. **Instrumentar** um fluxo com **OpenTelemetry** e achar a **etapa mais lenta**.

O problema: "está lento" — mas onde?

Uma requisição atravessa **vários serviços**. Se você só olha um por vez, cada um parece "ok" — e ninguém vê o conjunto.



"a resposta demora 2s" — mas **qual** etapa consumiu esse tempo?

💡 **Em miúdos:** é uma **encomenda que atrasou** passando por vários centros de distribuição. Sem rastreio, você não sabe **em qual** ela ficou parada.

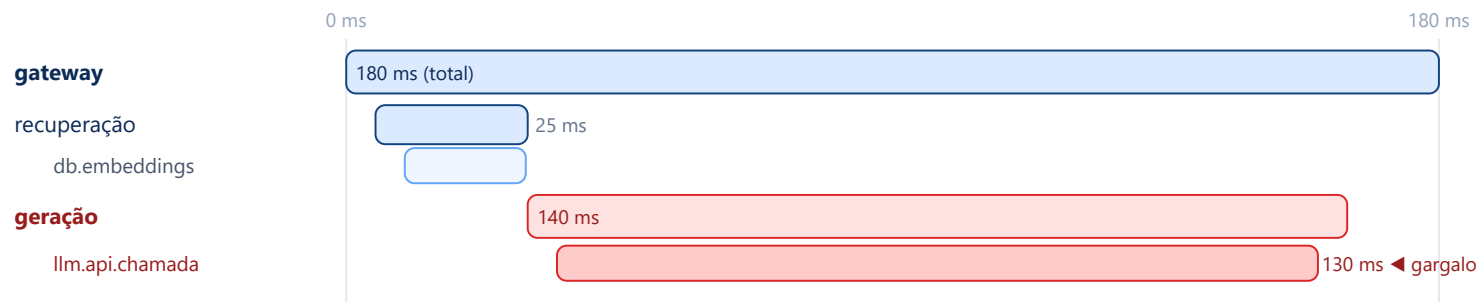
Os três pilares da observabilidade

pilar	o que é	responde
Logs	registros de eventos (texto, com hora)	" o que aconteceu neste ponto?"
Métricas	números agregados no tempo (req/s, latência, erros, CPU)	" quanto / como está a saúde geral?"
Traces	a jornada de UMA requisição pelos serviços	" onde ela passou e quanto gastou em cada etapa?"

💡 **Em miúdos:** **log** = o diário ("às 10h deu erro X"); **métrica** = o painel do carro (velocidade, temperatura); **trace** = o GPS de uma viagem específica (por onde passou e quando).

Rastreamento distribuído: o `trace_id` costura a jornada

Cada requisição recebe um `trace_id` único, propagado a **todos** os serviços. Cada etapa vira um **span** (nome + início + duração). Juntando os spans pelo `trace_id`, monta-se a **linha do tempo** (waterfall).



💡 Na "cascata", bate o olho: a **chamada ao LLM** domina (130 de 180 ms). É exatamente o que o `tracing_demo.py` imprime.

OpenTelemetry: instrumente uma vez

OpenTelemetry (OTel) é o **padrão aberto** do mercado para gerar logs, métricas e traces. Você **instrumenta o código uma vez** e **exporta** para o visualizador que quiser (Jaeger, Grafana, Datadog...), sem ficar preso a um fornecedor.

- Bibliotecas prontas para **FastAPI, requests, etc.** (instrumentação quase automática).
- O **contexto** (o `trace_id`) é propagado **entre serviços** nos cabeçalhos HTTP.

💡 **Em miúdos:** é uma **tomada padrão**: você liga seu código nela uma vez e pluga **qualquer** painel na outra ponta.

Laboratório

Rastrear uma requisição de ponta a ponta e achar o gargalo

Mão na massa — siga o Guia do Laboratório

Hoje são **4 missões**: entender o trace no aquecimento, **instrumentar** o fluxo com **OpenTelemetry**, **visualizar** no Jaeger e **achar o gargalo** — depois instrumentar o C3.A2.

👉 **Abrir o Guia do Laboratório »**

no índice do site: Aula 15 → **Guia do Lab**

💡 Aquecimento pronto e testado: `exemplos/aula15/tracing_demo.py` — monta um trace de exemplo e aponta a etapa mais lenta.

O que o exemplo mostra (aquecimento)

```
python tracing_demo.py
# - POST /pergunta (gateway)    180 ms #####
#   - recuperacao.buscar        25 ms ##
#   - geracao.responder         140 ms #####
#     - llm.api.chamada          130 ms #####
# Etapa mais lenta: 'llm.api.chamada' = 130 ms (72% dos 180 ms totais).
```

💡 A cascata mostra **onde** o tempo foi gasto. No lab, você gera isso a partir de código **real** com OpenTelemetry.

No seu trabalho — C3.A2

- **Instrumente** o fluxo do RAG com **OpenTelemetry** e colete um **trace completo** de uma pergunta.
- **Identifique o gargalo** (quase sempre a **chamada ao LLM** — e é por isso que ela tem o circuit breaker da Aula 11).
- Entregue uma **imagem do rastreamento** (Jaeger/console) e uma **análise** do gargalo.

💡 Puxa para o kit: comece instrumentando o **gateway** e a **geração**. Propague o contexto entre eles para o trace "costurar" os dois serviços.

Atividade para casa

1. **Instrumente** o **projeto inteiro** (C3.A2) com OpenTelemetry (gateway + serviços).
2. **Colete** um trace de ponta a ponta e **salve a imagem** do rastreamento.
3. **Escreva** a análise: qual é a **etapa mais lenta** e **o que** você faria para melhorá-la.

📌 **Entregar até a próxima aula (EAD de Segurança):** o projeto instrumentado + a imagem do rastreamento + a análise do gargalo. Roteiro no **Guia do Laboratório**.

◆ Foco ENADE

O que costuma cair:

- Os **três pilares**: logs × métricas × traces (o que cada um responde).
- **Rastreamento distribuído** e o papel do `trace_id` / span / contexto propagado.
- **OpenTelemetry** como padrão aberto (instrumenta uma vez, exporta para vários).
- Usar o trace para **localizar o gargalo** (a etapa mais lenta).

Termos-chave: Observabilidade · Log · Métrica · Trace · Span · trace_id · OpenTelemetry · Gargalo

💡 Pegadinha: confundir **log** (evento pontual) com **métrica** (número agregado) e com **trace** (jornada de uma requisição específica).

Questão de autoavaliação (estilo ENADE)

Uma requisição passa por **quatro** microsserviços e os usuários relatam lentidão **intermitente**. A equipe quer descobrir **em qual serviço** o tempo é gasto **em cada requisição específica**. O recurso de observabilidade mais adequado é:

- A) apenas **logs** de cada serviço, lidos separadamente
- B) apenas **métricas** agregadas de CPU
- C) **rastreamento distribuído (traces)** com um `trace_id` propagado entre os serviços
- D) aumentar o nível de log para DEBUG em produção
- E) reiniciar os serviços periodicamente

Resolução — alternativa C

- Só o **trace** liga as etapas de **uma mesma requisição** (via `trace_id`) e mostra **onde** o tempo foi gasto.
- **Logs** soltos (A) não costuram a jornada; **métricas** (B) dão o agregado, não a requisição específica; (D) e (E) não localizam o gargalo.

💡 Para "onde está lento nesta requisição?", a resposta é quase sempre **tracing distribuído**.

Fora da sala · Glossário

Para estudar

- Docs do **OpenTelemetry** (conceitos: trace, span, context).
- **Jaeger** (visualizador de traces) — tutorial de 5 min.
- Rode o `tracing_demo.py` mudando as durações e veja o gargalo mudar.

Glossário

- **Observabilidade:** capacidade de entender o estado interno pelo que o sistema emite.
- **Log:** registro de um evento.
- **Métrica:** número agregado no tempo.
- **Trace:** jornada de uma requisição; composto de **spans**.
- **trace_id:** identificador que costura a jornada entre serviços.
- **OpenTelemetry:** padrão aberto de instrumentação.

Até a próxima aula 🚀

Instrumente o C3.A2, colete um trace e analise o gargalo.

Próxima (Aula 16 · EAD): Segurança, privacidade e LGPD — proteger a plataforma (TLS, autenticação/JWT, autorização, LGPD) + revisão para o ENADE.

← Anterior

Aula 14 · Orquestração

≡ Índice

todas as aulas